

 Tags:

Submit Codes AI Code Tips

给出两个正整数a和b，计算a+b的值。

第一行是一个数字T，代表有T组输入。

接下来有T行，每行代表一组输入，每组包含两个不超过1000位的正整数a和b。

对于每组输入，输出两行，第一行为"Case i:"（不包含双引号，其中i为第i组测试数据），第二行为a+b的值，每组测试数据之后加一个空行。

```
input Copy
2
1 5
10000000000000000000000000000000 10000000000000000000000000000000

output Copy
Case 1:
6

Case 2:
20000000000000000000000000000000
```

如果做过前面的题目，此题在输入输出格式上不会碰到啥问题，相较于Lecture (1) 的格式，只是多了测试数据的计数而已和最后的回车而已。本题的主要问题在于如何储存长达1000位的整数并让它们相加，显然使用int或long long都没法存下这个“庞然大物”。此时我们需要另寻它法。如果你用的是Java，那么本题你可以使用Java内置的BigInteger类轻松解决，如果是C/C++，那么本题只能使用长度为1000的数组来储存这个数字了，为了方便读入，可以使用字符串来储存这些数字，并模拟加法使他们相加。这里还需要注意，a和b都不超过1000位，但它们相加的值可能超过1000位，因此在用C风格字符串时，需

要格外注意数组是否足够大。
因此我们可以写出如下代码：

C语言版：

```
1#include<stdio.h>
2#include<string.h>
3int main() {
4    int i, t, T, aLen, bLen, add, sum;
5    char a[1002], b[1002], c[1002], temp;
6    scanf("%d", &T);
7    for (t = 1; t <= T; t++) {
8        scanf("%s %s", a, b);
9        // 获取数字长度
10       aLen = strlen(a);
11       bLen = strlen(b);
12       // 将数字反转，使其从个位开始对齐
13       for (i = 0; i < aLen / 2; i++) {
14           temp = a[i];
15           a[i] = a[aLen - i - 1];
16           a[aLen - i - 1] = temp;
17       }
18       for (i = 0; i < bLen / 2; i++) {
19           temp = b[i];
20           b[i] = b[bLen - i - 1];
21           b[bLen - i - 1] = temp;
22       }
23       add = 0;
24       // 开始模拟加法
25       for (i = 0; i < aLen || i < bLen; i++) {
26           if (i < aLen && i < bLen)
27               sum = a[i] - '0' + b[i] - '0' + add;
28           else if (i < aLen)
29               sum = a[i] - '0' + add;
30           else if (i < bLen)
31               sum = b[i] - '0' + add;
32           add = 0;
33           if (sum > 9) {
34               add = 1;
35               sum -= 10;
36           }
37           c[i] = sum + '0';
38       }
39       // 若加完后还有进位，则补上
40       if (add) c[i++] = add + '0';
41       // 逆序输出相加的和
42       printf("Case %d:\n", t);
43       for (i--; i >= 0; i--)
44           printf("%c", c[i]);
45       puts("\n");
46   }
47   return 0;
48 }
```

C++版:

```
1#include<iostream>
2#include<algorithm>
3#include<string>
4using namespace std;
5int main() {
6    int i, T, aLen, bLen, add, sum;
7    string a, b, c;
8    cin >> T;
9    for (int t = 1; t <= T; t++) {
10        cin >> a >> b;
11        // 获取字符串长度
12        aLen = a.length();
13        bLen = b.length();
14        // 反转a和b, 使其从个位开始对齐
15        reverse(a.begin(), a.end());
16        reverse(b.begin(), b.end());
17        c = ""; // c=a+b
18        add = 0; // 进位
19        // 开始模拟加法
20        for (i = 0; i < aLen || i < bLen; i++) {
21            if (i < aLen && i < bLen)
22                sum = a[i] - '0' + b[i] - '0' + add;
23            else if (i < aLen)
24                sum = a[i] - '0' + add;
25            else if (i < bLen)
26                sum = b[i] - '0' + add;
27            add = 0;
28            if (sum > 9) {
29                add = 1;
30                sum -= 10;
31            }
32            c += sum + '0';
33        }
34        // 若加完后还有进位, 则补上
35        if (add) c += add + '0';
36        // 逆序并输出
37        reverse(c.begin(), c.end());
38        cout << "Case " << t << ":" << endl;
39        cout << c << endl << endl;
40    }
41    return 0;
42}
```

Java版:

```
1import java.math.BigInteger;
2import java.util.Scanner;
3public class Main {
4    public static void main(String[] args) {
5        int T;
6        BigInteger a, b;
7        Scanner in = new Scanner(System.in);
8        T = in.nextInt();
9        for (int t = 1; t <= T; t++) {
10            a = new BigInteger(in.next());
11            b = new BigInteger(in.next());
```

```
12 System.out.println("Case " + t + ":");
13 System.out.println(a.add(b) + "\n");
14 }
15 in.close();
16 }
17 }
```

Author

CHEN, Yupeng

Submit

Codes

HZNUOJ is based on [HUSTOJ](#)

--- [Maintainer List](#) ---

Server Time: 2025-8-26 14:56:00